Projeto Integrador - Criptografia, Banco de dados e desenvolvimento seguro.

Empresa On.loc

A On.loc funciona com o empréstimo de equipamentos - registrados com um número de patrimônio - para outras empresas de pequeno, médio e grande porte.

Dor resolvida

O que estamos proponto fazer é um sistema de vistoria de equipamentos. Nesse sistema, fotos e dados serão anexadas para que uma avaliação dos disposivitos possa ser realizada.

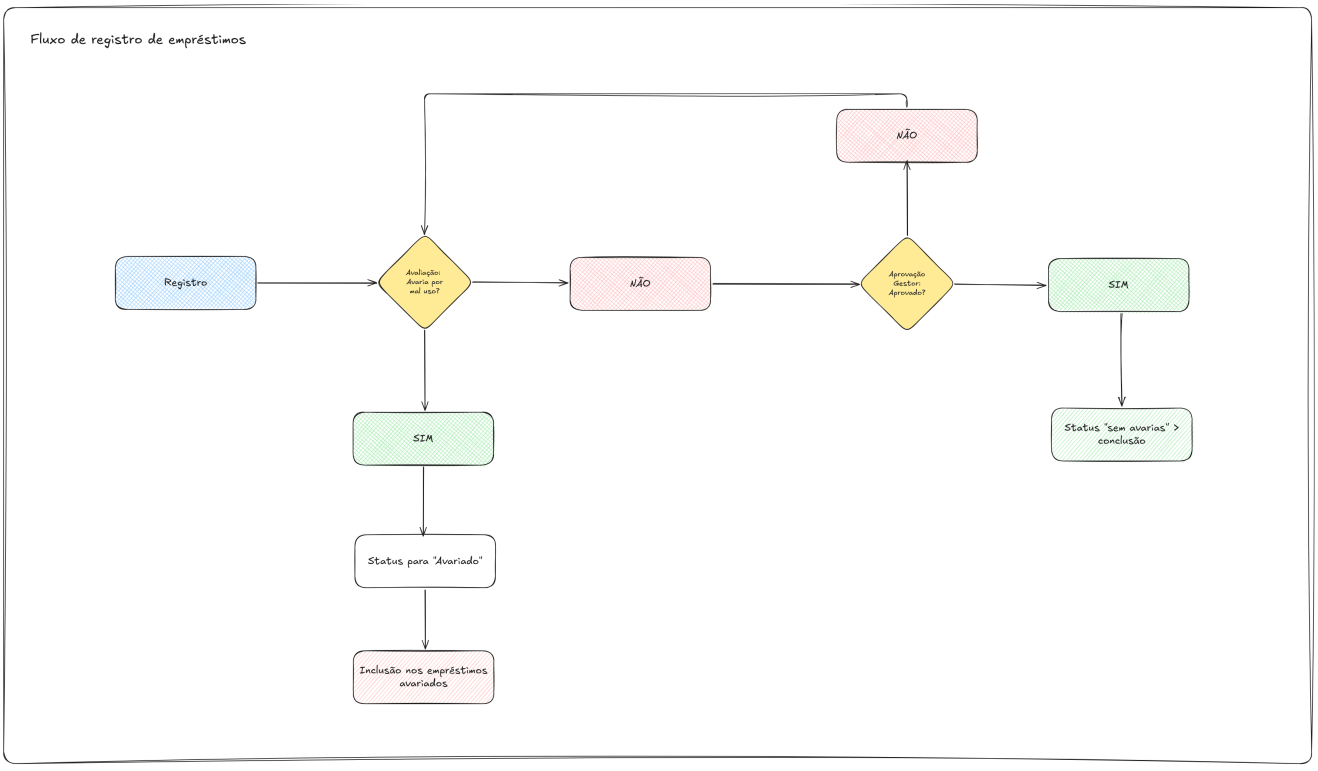
Esse sistema deve armazenar fotos das máquinas, número de série, número de patrimônio com todos os sub-itens emprestados.

O agrupamento deve ser feito por cliente, deve registrar dia e horário, deve conter os dados do cliente como CPF/CNPJ, nome fantasia da empresa e localização.

A ideia se baseia em:

- **Registrar** os equipamentos devolvidos (relacionando a empresa e equipamentos)
- **Avaliar** os equipamentos, buscando alguma avaria por **mal uso**.
- **Aprovar** a auditoria e "arquivar" a devolução, sendo possível consultar cada equipamento registrado.

Fluxo proposto



Detalhamento técnico

Banco de dados

Mysql conteinerizado com docker. Usuários de grupos sem permissão de root.
Para o armazenamento de imagens usaremos uma instância do MinIO (open source S3 bucket). Assim, contaremos com o armazenamento da VPS para armazenar os arquivos.

Teremos tabelas para:

-> Usuários (Nome, email, enum ('Analista', 'Gestor'))
-> Senhas (criptografadas por sinal, one to one com usuários)
-> Clientes registrados (responśavel, cpf/cnpj, nome fantasia e localização)
-> Registros de empréstimos (data, organização, detalhes adicionais, relação entre clientes e empréstimos realizados, status do empréstimo (sem avarias, com avarias))
-> Empréstimos realizados (cada equipamento com número de patrimônio)
-> Empréstimos avariados (relação entre registros de empréstimos, emrpréstimo realizado e avaria localizada).

Backend

-> Será feito em C++ puro.
-> Adicionaremos uma camada com o "libmicrohttpd" para abrir conexões http.
-> Para conexão com o BD utilizaremos o `MySQL Connector/C++`;
-> Testes unitários feitos em Gtest (agnóstico à ecossistema).

Frontend

-> Feito com Angular - Javascript.
-> Conexões com backend feitas a partir de módulos internos (angular-http)
-> Testes unitários com Jest
-> Testes de interface com Playwright.

Deployment

-> VPS na hazelnet (qualquer uma barata para podermos iniciar) - pelo menos 20gb de armazenamento para podermos armazenar os arquivos.
-> Proxy reverso configurado com nginx, apontando para instâncias internas no docker
-> SSL gerado com let's encrypt
-> Domínio genérico para podermos apontar nossa aplicação - configuração no squarespace ou qualquer um similar.

Criptografia

Onde iremos implementar?

-> Tabela de senhas totalmente criptografadas (nada em plain text).
-> JSON Web Token (JWT) para validação de acesso recorrente - apenas 1 hora de sessão ativa.

Estratégias de Testes Preliminares

A estratégia de testes preliminares para a On.loc tem como foco assegurar a integridade do fluxo de vistoria, a confiabilidade da persistência dos dados e a segurança das credenciais e arquivos anexados antes da liberação do MVP. No nível unitário, a camada de backend em C++ será validada com o Google Test (GTest), concentrando-se na formatação correta de payloads e no tratamento de erros via `libmicrohttpd`. Paralelamente, os componentes e serviços do frontend em Angular passarão por testes com Jest, cobrindo validações essenciais de formulários — como máscaras e regras de unicidade para CPF/CNPJ —, interações visuais e o comportamento de interceptors HTTP que gerenciam os tokens JWT de uma hora.

Os testes de integração atuarão na comunicação entre os contêineres e serviços externos, garantindo que o MySQL Connector/C++ opere de forma confiável sob usuários sem privilégios de root e que o MinIO processe com sucesso o upload e o resgate de imagens em alta resolução. Esse nível de verificação também valida o tratamento de falhas transacionais, impedindo que fotos fiquem órfãs no bucket S3 caso a gravação do laudo de avaria falhe no banco de dados.

Na fase de testes ponta a ponta (E2E) com Playwright, será simulada a jornada completa do usuário final no navegador, percorrendo o login autenticado, a seleção do cliente, a conferência de sub-itens emprestados, o upload das evidências fotográficas e a classificação do status entre devoluções com ou sem avarias.

Por fim, os testes preliminares de segurança e infraestrutura verificarão o isolamento do proxy reverso Nginx com terminação SSL via Let's Encrypt, a correta expiração das sessões JWT, a proteção contra injeções nas rotas expostas e o controle de acesso baseado em papéis (RBAC), assegurando que analistas possam registrar vistorias, mas que o arquivamento e a aprovação formal da auditoria fiquem estritamente restritos ao perfil de gestor.

Planejamento

Datas	
Semana 14/09 - 18/09	Entrega da carta com o Planejamento. Discussão e aprovação das ideias propostas.
Semana 21/09 - 25/09	Início do desenvolvimento. Contratação de VPS, configuração de serviços, instâncias e backbone das aplicações front e back.
Semana 12/10 - 16/10	Apresentação MVP.
Semana 26/10 - 30/10	Produto funcional.
Semana 02/10 - 13/10	Testes e verificação de requisitos funcionais. Implementação de testes e2e.

Riscos e Premissas

O projeto fundamenta-se na premissa de que a infraestrutura inicial de 20 GB de armazenamento na VPS será suficiente para comportar o sistema operacional, os contêineres Docker, o banco de dados MySQL e o armazenamento de imagens no MinIO durante a fase de testes e validação do MVP. No âmbito da segurança e do negócio, assume-se que as instâncias rodarão estritamente com usuários sem privilégios de root sob a proteção do proxy reverso Nginx com certificados TLS válidos, e que todos os equipamentos físicos devolvidos e clientes cadastrados contarão com identificadores legíveis e oficiais, como números de patrimônio e CPF/CNPJ válidos. Por fim, assume-se que a alocação de tempo planejada viabilizará a prontidão dos serviços e do backbone de rede a tempo da entrega funcional.

Em contrapartida, o principal risco técnico reside no esgotamento prematuro do disco da VPS decorrente do upload massivo de fotografias em alta resolução no MinIO, o que pode paralisar a gravação no MySQL e derrubar o ambiente. Para conter essa ameaça, aplicaremos compressão de imagens no frontend antes do envio e configurar alertas rigorosos de uso de partição. Há também o risco de inconsistências transacionais entre o banco e o bucket S3, gerando laudos de avaria sem comprovação visual ou imagens órfãs no MinIO em caso de falha de conexão, cenário que exige rotinas de reversão automática caso o commit relacional falhe.

Soma-se a isso a complexidade de implementar o gerenciamento de memória manual em C++, onde vazamentos de ponteiros, falhas de sincronismo ou erros na validação de chaves podem corromper auditorias e provocar paradas inesperadas do serviço; esse ponto será mitigado por meio de profilings com AddressSanitizer e baterias exaustivas de testes unitários no GTest. Por fim, identificam-se riscos de usabilidade e compatibilidade na interface Angular durante a captura e envio de arquivos por múltiplos dispositivos móveis,

cuja mitigação dependerá de validações antecipadas de layout e comportamento com Playwright.